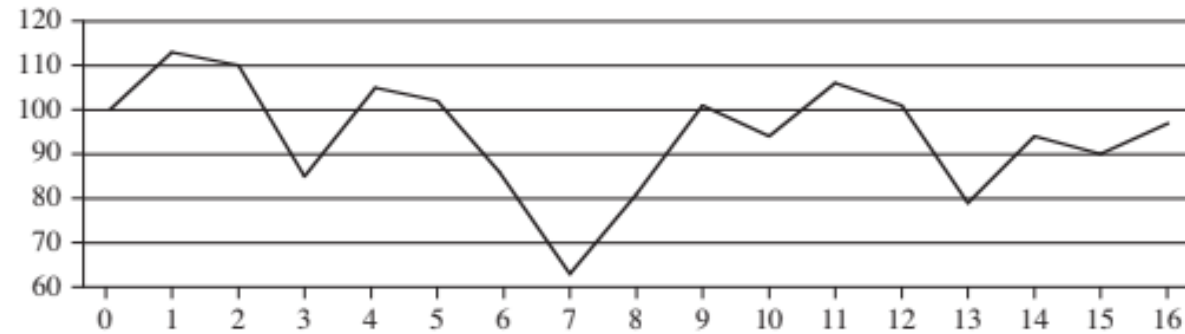


The Maximum-Subarray Problem:

- ① To invest in the Volatile Chemical Corporation
- ② The stock price of the Volatile Chemical Corporation is rather volatile.
- ③ Condition: To buy one unit of stock only one time and then sell it at a later date, buying and selling after the close of trading for the day.
- ④ To compensate for this restriction, you are allowed to learn what price of the stock will be in future.
- ⑤ Goal: To maximize profit. $\rightarrow$ (buy low and sell high)

$\hookrightarrow$ "you might not be able to buy at the lowest price and then sell at highest price."

Figure: the lowest price occurs after day 7, which occurs after the highest price, after day 1.



Day	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Price	100	113	110	85	105	102	86	63	81	101	94	106	101	79	94	90	97
Change		13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

→ We would maximize profit by buying the lowest price after day ~~0~~ 7. IF this strategy always worked, then it would be easy to determine how to maximize profit: find the highest and lowest price.

Figure: An counter example.

A brute-force solution:

- Just try every possible pair of buy and sell dates in which the buy date precedes the sell date.
- A period of n days has $\boxed{nC_2}$ such pairs of dates. Since nC_2 is $\Theta(n^2)$, and best we hope for is to evaluate each pair of dates in constant time.

Transformation:

- We want to find a sequence of days over which the net change from the first day to the last is maximum.
- Instead of looking at the daily price, let us consider daily change in price.
 $\left\{ \begin{array}{l} \Rightarrow \text{Change on day } i \text{ is the difference between the price after day } i-1 \\ \text{and after day } i \end{array} \right\}$

* We now want to find the nonempty, contiguous subarray of A whose values have the largest sum. We call this contiguous subarray the "maximum subarray".

* In the array of figure, the maximum subarray of $A[1...16]$ is $A[8...11]$, with the sum 43. Thus, you would want to buy the stock just before day 8 and sell it after day 11 earning a profit of $\boxed{\$43 \text{ per share}}$.

*=

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

maximum subarray

* we still need to check $n-1 C_2 = \Theta(n^2)$ subarray for a period of n days.

Maximum-sum contiguous subarray

Given an array $A[]$ of n numbers, find the maximum sum found in any contiguous subarray

A zero length subarray has maximum 0

Example:

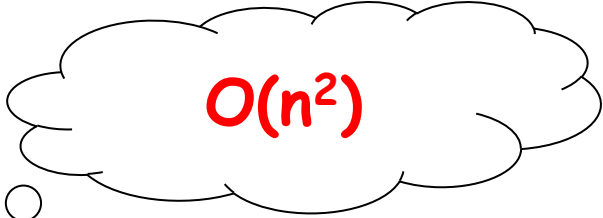
- if $A[]$ contains **-2, 5, -3, 6, -1**, then the maximum sum will be 5-3+6 = **8**
- if $A[]$ contains **10, 15, -3, -4, -2, -1, 8, 5**, then the maximum sum will be **28** (all numbers added together)

$O(n^2)$, $O(n \log n)$, $O(n)$ time algorithms

Brute-force algorithm

Brute-force algorithm

- All possible contiguous subarrays
 - $A[1..1]$, $A[1..2]$, $A[1..3]$, ..., $A[1..(n-1)]$, $A[1..n]$
 - $A[2..2]$, $A[2..3]$, ..., $A[2..(n-1)]$, $A[2..n]$
 - ...
 - $A[(n-1)..(n-1)]$, $A[(n-1)..n]$
 - $A[n..n]$
- How many of them in total?
- Algorithm: For each subarray, compute the sum. Find the subarray that has the maximum sum.



$O(n^2)$

Idea of brute-force algorithm

Example:

sum from A[1]:

2 -6 -1 3 -1 2 -2
 2 -4 -5 -2 -3 -1 -3 → max: 2 j=1 to 7

sum from A[2]:

-6 -7 -4 -5 -3 -5 j=2 to 7

sum from A[3]:

-1 2 1 3 1 j=3 to 7

sum from A[4]:

3 2 4 2 j=4 to 7

sum from A[5]:

-1 1 -1 j=5 to 7

sum from A[6]:

2 0 j=6 to 7

sum from A[7]:

-2 j=7

Brute-force algorithm

- Outer loop: index variable i to indicate start of subarray, for $1 \leq i \leq n$, i.e., $A[1]$, $A[2]$, ..., $A[n]$
 - for $i = 1$ to n do ...
- Inner loop: for each start index i , we need to go through $A[i..i]$, $A[i..(i+1)]$, ..., $A[i..n]$
 - use an index j for $i \leq j \leq n$, i.e., consider $A[i..j]$
 - for $j = i$ to n do ...

Pseudo code

```

sum = 0
max = 0
* for i = 1 to n do
  begin
    sum = 0
    for j = i to n do
      sum = sum + A[j]
      if sum > max
        then max = sum
    end
  end
end

```

[2]

A = $\begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 2 & -6 & -1 & 3 & -2 \end{bmatrix}$

loop inside loop
 n^2

for i = 1 to 5

Time complexity?

$O(n^2)$
 { j = 1
 sum = sum + A[1] = 2 } max = 2

if sum > max
 2 > 0

max = 2

j = 2
 sum = -4

j = 3 sum = -5
 j = 4 sum = -2
 j = 5 sum = -7

for i = 4
 sum = 0
 j = 4 to 5
 sum = 3
 3 > 2

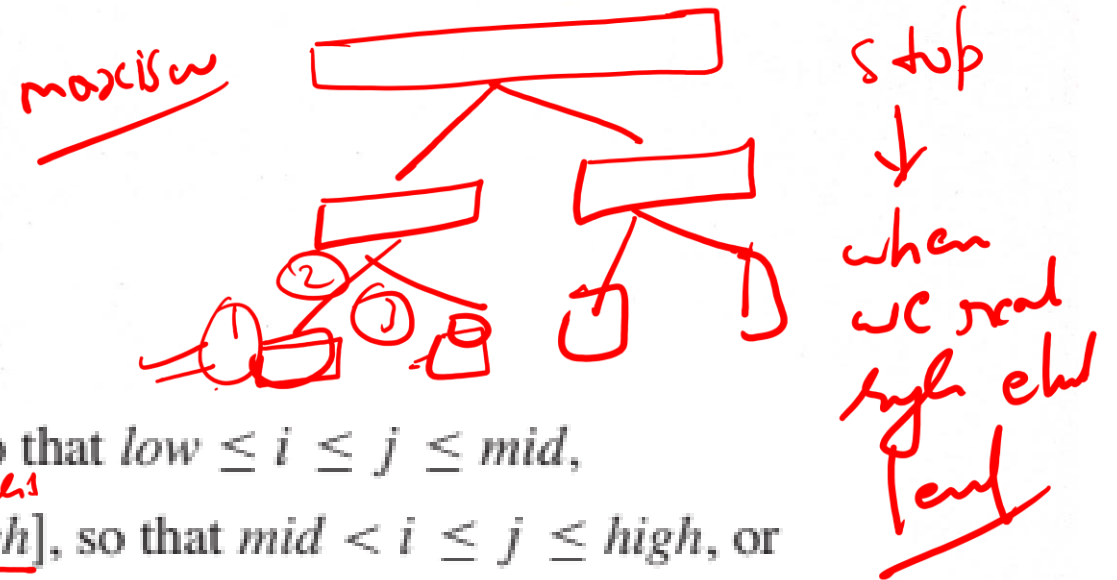
Algorithmic Paradigm Δ Brute Force: $\rightarrow$ Explore all possibilities and identify the solution irrespective using any prior knowledge.
 $\hookrightarrow \textcircled{n^2}$

Divide-and-conquer algorithm

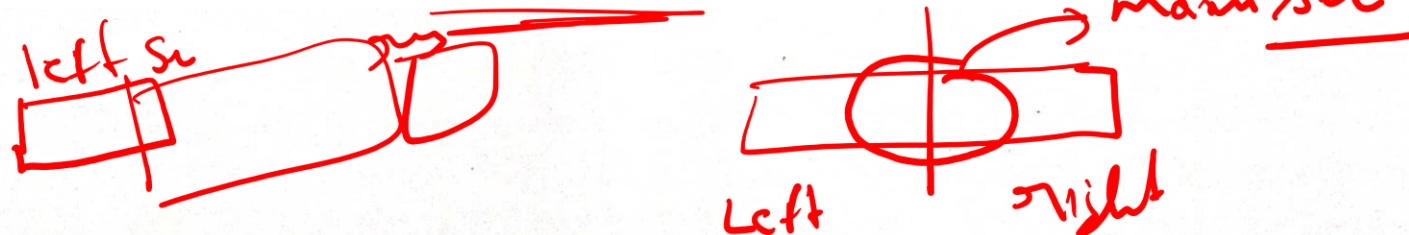
main subproblem $\hookrightarrow O(n \log n) \rightarrow$ merge sort / quick sort

"The maximum sub-array problem is interesting only when the array contains some negative numbers. If all the array entries were non-negative, then the maximum subarray problem would present no challenge, since the entire array would give the greatest sum"

Divide and Conquer Solution :-



- entirely in the subarray $A[\text{low} \dots \text{mid}]$, so that $\text{low} \leq i \leq j \leq \text{mid}$,
- entirely in the subarray $A[\text{mid} + 1 \dots \text{high}]$, so that $\text{mid} < i \leq j \leq \text{high}$, or
- crossing the midpoint, so that $\text{low} \leq i \leq \text{mid} < j \leq \text{high}$.



$$A = \begin{bmatrix} \overset{1}{-2} & \overset{2}{+7} & \overset{3}{+9} & \overset{4}{-3} & \overset{5}{-1} & \overset{6}{2} & \overset{7}{-2} \end{bmatrix}$$

$A[2,7] = 16$

$\lceil n/2 \rceil$

$A[3,3]$

1	9
---	---

$A[2,3] = 16$

$A[6,6] = 2$

$[-2 \ 7 \ 9 \ -3]$

$[-1 \ 2 \ -2]$

$A[2,2] = 16$

$A[3,3] = 9$

$A[5,5] = 2$

$A[7,7] = -2$

$A[2,2] = 7$

$[-2 \ 7]$

$[9 \ -3]$

$[-1, 2]$

$[-2]$

2	2	=0
6	7	

$A[2,6]$

5
-2
7

1 $[-2]$

5

2 $[7]$

3 $[9]$

4 $[-3]$

5 $[-1]$

6 $[2]$

$A[1,1] = -2$

$A[2,2] = 7$

2	3	4	5	6
7	9	-3	-1	2

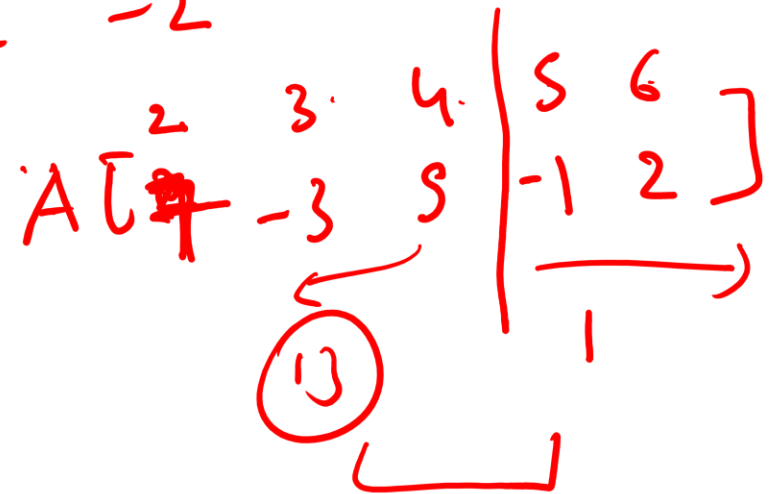
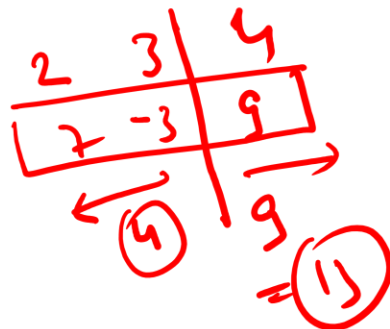
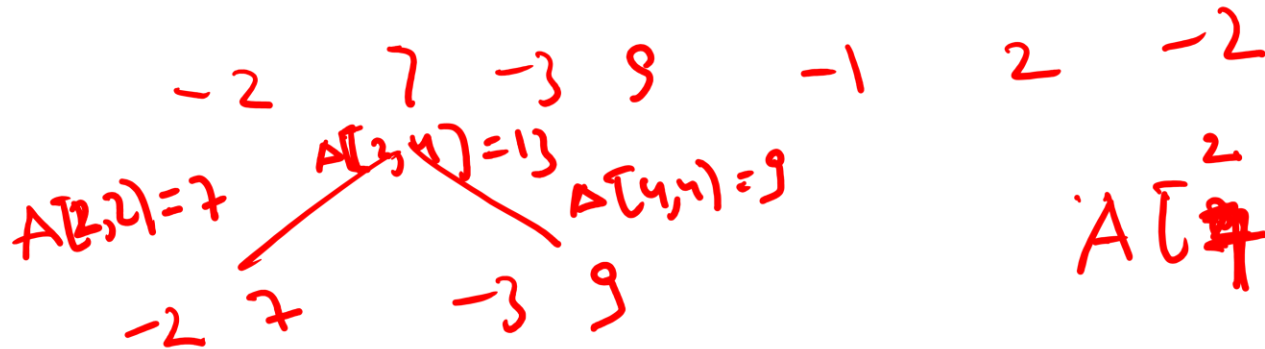
(13) (14) 1

¹
-2 ²
7 ³
-3 9 -1 ⁶
2 ¹
-2

$\Delta[2,6] = \textcircled{14}$

$\Delta[2,4] = \textcircled{13}$

$\Delta[6,6] = \textcircled{2}$



$\Delta[2,6] = \textcircled{14}$

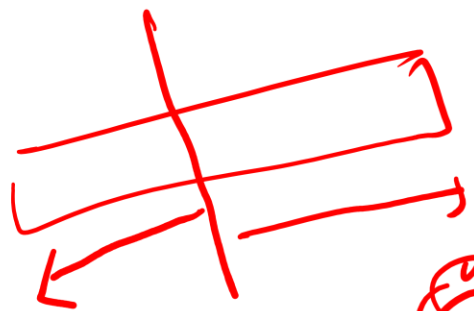
FIND-MAXIMUM-SUBARRAY (A, ²low, [~]high)

```
1  if high == low
2      return (low, high, A[low]) // base case: only one element
3  else mid =  $\lfloor (low + high) / 2 \rfloor$ 
4      (left-low, left-high, left-sum) =
        FIND-MAXIMUM-SUBARRAY(A, low, mid)
5      (right-low, right-high, right-sum) =
        FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
6      (cross-low, cross-high, cross-sum) =
        FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
7      if left-sum  $\geq$  right-sum and left-sum  $\geq$  cross-sum
8          return (left-low, left-high, left-sum)
9      elseif right-sum  $\geq$  left-sum and right-sum  $\geq$  cross-sum
10         return (right-low, right-high, right-sum)
11     else return (cross-low, cross-high, cross-sum)
```

$\Rightarrow$ $(2, ?)$

log(n) nlg

$n/2 + n/2$
 $= 2$



FIND-MAX-CROSSING-SUBARRAY(*A*, *low*, *mid*, *high*)

```

1  left-sum = -∞
2  sum = 0
3  for i = mid downto low
4      sum = sum + A[i]
5      if sum > left-sum
6          left-sum = sum
7          max-left = i
8  right-sum = -∞
9  sum = 0
10 for j = mid + 1 to high
11     sum = sum + A[j]
12     if sum > right-sum
13         right-sum = sum
14         max-right = j
15 return (max-left, max-right, left-sum + right-sum)

```

$T(n) = 2(T(n/2)) + 2$

$A \left[\begin{smallmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{smallmatrix} \right]$

$\left[\begin{smallmatrix} 3 & 4 & 5 & 6 & 7 \end{smallmatrix} \right]$

$n/2$

2	3	4	5	6	7	8	9
-4	-5	-7	-6	7	9	-6	-1

$\left[\begin{smallmatrix} 5 & 6 & 7 & 8 \end{smallmatrix} \right]$

$n/2$

4	5	6	7	8	9
-6	20	20	-2	0	-9

$\left[\begin{smallmatrix} 5 & 6 & 7 & 8 \end{smallmatrix} \right]$

$\left[\begin{smallmatrix} 5 & 6 & 7 & 8 \end{smallmatrix} \right]$

$$T(n) = 2T(n/2) + n$$

↳ merge sort

↳ $O(n \lg n)$

Brute force $\rightarrow (n^2) \rightarrow n \lg n$

Kadane's $\rightarrow \textcircled{n}$

DP

Example:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 \\ 13 & -3 & -25 & 20 & -3 & -16 & -23 & 18 & 20 & -7 & 12 & -5 & -22 & 15 & -4 & 7 \end{bmatrix}$$

mid = 8

= FIND-MAXIMUM-SUBARRAY (A, 1, 8)

$$A' = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 13 & -3 & -25 & 20 & -3 & -16 & -23 & 18 \end{bmatrix}$$

mid = 4

= FIND-MAXIMUM-SUBARRAY (A', 1, 4)

$$A'_1 = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 13 & -3 & -25 & 20 \end{bmatrix}$$

mid = 2

$$A'_{11} = \begin{bmatrix} 1 & 2 \\ 13 & -3 \end{bmatrix}$$

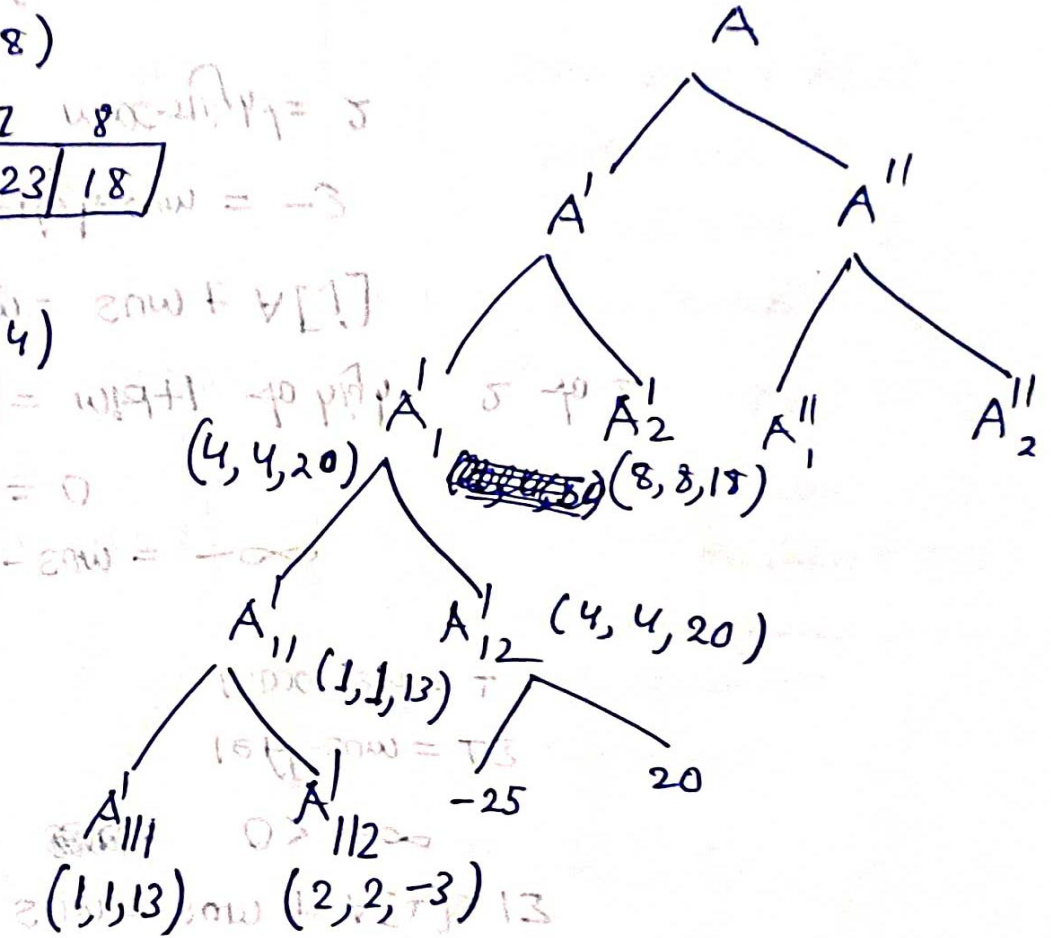
mid = 1

$A'_{111} = [13]$ Here, $\boxed{\text{high} = \text{low}}$

return (low, high, A(low))

= (1, 1, 13)

→ FIND-MAX-CROSSING-SUBARRAY (A', low, mid, high)



(b) A, hold (not 976)

$$w!g = J$$

Wig = 3

$$= \text{sum} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$
$$= \text{EIND} - \text{WAK} \cdot \text{WOM} - \text{ZINS} \cdot \text{KAP} \cdot (Y' \cdot T' \cdot \delta)$$

Wg. 8

$Y = [13 \quad -2 \quad -32 \quad 30 \quad -3 \quad -19 \quad -33 \quad 18 \quad 50 \quad -1 \quad 15 \quad -2 \quad -35 \quad 12 \quad -4 \quad 1]$

$$\text{Sum} = 0$$
$$\text{sum} = \text{sum} + A[1] = 13$$
$$\text{left-sum} = 13$$
$$\text{max-left} = 1$$
$$\text{sum} = 0$$
$$\text{sum} = \text{sum} + A[j]$$
$$\text{max-right} = 2$$

return (1, 2, 13 + (-3))

метод $(1, 2, 10)$

$cross-low = 1$
 $cross-high = 2$
 $cross-sum = 10$

return (1, 1, 13)

similarly $A'_{12} \leftarrow (4, 4, 20)$

Now, A'_1

FIND-MAX-CROSSING-SUBARRAY
 $(A'_1, 1, 2, 4)$

* $A'_1 = (4, 4, 20)$ $A'_2 = (8, 8, 18)$

↳ FIND-MAX-CROSSING-SUBARRAY($A'_1, 4, 4, 8$)

$\left\{ \begin{array}{l} max-left = 4, left-sum = 20 \\ max-right = 8, right-sum = -24 \end{array} \right.$

$left-sum + right-sum = -16$

$left-sum = -\infty$
 $sum = 0$
 for $i = 2$ to 1

$sum = sum + A[2]$

$sum = -3$

$left-sum = -3$

$max-left = 2$

↳ $sum = -3 + 3 = 10$

$left-sum = 10$

$max-left = 1$

$max-right = 4$

$right-sum = -5$

return (1, 4, 5)

Analyzing the divide-and-conquer algorithm:

④

- > For starters, line 1 take constant time. The base-case, when $n=1$, is easy:
line 2 takes constant time, and so

$$T(1) = \Theta(1)$$

- > The recursive case occurs when $n > 1$. Lines 1 and 3 take constant time.

- > Each of the subproblems solved in lines 4 and 5 is on a subarray of $n/2$ elements

- > Note:- Our assumption that the original problem size is a power of 2 ensures that $n/2$ is an integer

- > So we spend $T(n/2)$ time solving each of them.

- > Because we have to solve two subproblems - for the left subarray and for the right subarray - the contribution to the running time from lines 4 and 5 $\rightarrow 2T(n/2)$

- ⑤
- > The call to FIND-MAX-CROSSING-SUBARRAY in line 6 takes $\Theta(n)$ times
 - > Lines 7-11 take only $\Theta(1)$ time. For the recursive case, therefore, we have

$$\begin{aligned} T(n) &= \Theta(1) + 2T(n/2) + \Theta(n) + \Theta(1) \\ &= 2T(n/2) + \Theta(n) \end{aligned}$$

- > Combining above equation:-

$$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 2T(n/2) + \Theta(n) & \text{if } n>1 \end{cases}$$

- > This recurrence is the same as recurrence for merge sort.

$$\boxed{T(n) = \Theta(n \log n)}$$

